

# Lecture 7

Thursday February 12th 2025

## Quiz 1

- Thursday 19th covering sections 1- 6

## Recap

- Talked about processes
- went over examples using `fork()`
- Start/stop processes
- Context switching
- Process scheduling

## Three Level Process Scheduling

- Arriving job goes into a queue and waits for admission Scheduler to main memory (**Long-term**)
- CPU Scheduler for access to CPU (**Short-Term**)
- Memory Scheduler for access to the Disk (**Mid-Term**)
- Can have thousands of jobs waiting on a turn with the CPU, this is the short term scheduling

## Cooperating Processes

- **Independent process** cannot affect or be affected by the execution of another process
- **Cooperating process** can affect or be affected by the execution of another process
  - an independent process can not be interrupted by the OS so the program needs to be stopped
- Advantages of process cooperation
  - information sharing
  - computation speed-up
  - computation speed-up
  - modularity
  - Convenience
- Disadvantage
  - Synchronization issues and race conditions

## Inter-process Communication (IPC)

- Mechanism for processes to communicate and to synchronize their actions
- **Shared memory** uses the same address space and shared variables
- **Message passing** - processes communicate with each other without resorting to shared variables

## Example Problem

- watch the lecture to review
- Programs are showing inter-process communication
  - shared memory
  - message passing
  - same concepts as MPI from distributed systems projects

## Threads

- Threads allow us to work in better than sequential time
- Introduction of parallelism
- class example was summation of 10 numbers
  - $O(n)$  sequential
  - $O(\log n)$  when using parallelism to sum with a tree
- Classical view, one program runs inside a process and has:
  - single line of execution
  - works sequentially
- Program can use threads
  - does multiple work simultaneously
- Avoids blocking program due to slow I/O
- Think about an Email client
  - receive email
  - compose email
  - send email
  - calendar notification
- If the program is sequential
  - cannot write an email when your email client is receiving an email

## Single and multithreaded process

- each thread should have its own stack when working with multithreaded processes

## New Process Description Model

- Multithreading requires changes in the process description model
  - each thread of execution receives its own control block and stack

Look in to how to work with threads, processes, and forks

First project will be to write a simulation of a basic terminal

- if you enter cd it will move to a different directory